

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский
Томский политехнический университет

Инженерная школа информационных технологий и робототехники
Отделение информационных технологий

Отчет по лабораторной работе №20 по дисциплине

«Язык Kotlin и основы разработки»

Уведомления

Выполнил:

Студент группы 1А22



О.К. Кравцов

Проверил:

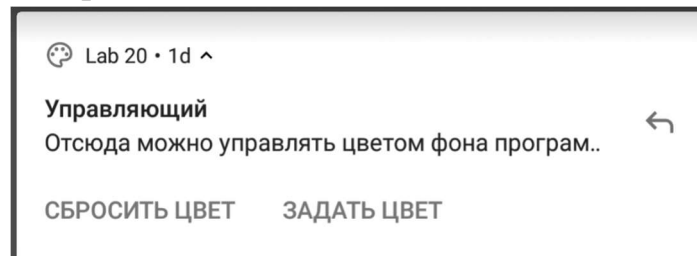
Ст. преп. ОИТ ИШИТР



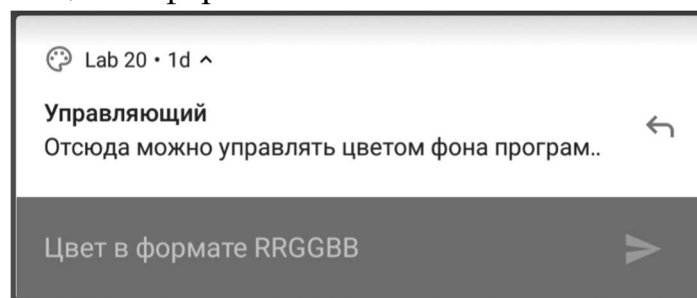
В.А. Дорофеев

Задание

Напишите программу, которая по нажатию кнопки отображает уведомление – конечно, с обязательной проверкой разрешена ли отправка уведомлений, запросом такого разрешения и т.п. формальностями. Уведомление должно выглядеть примерно следующим образом:



При нажатии уведомления должна запускаться программа (даже если она уже запущена, что поделать...). При нажатии кнопки «Сбросить цвет» – фон активности становится белым. При нажатии кнопки «Задать цвет» – появляется поле ввода, в котором можно ввести цвет в формате RRGGBB:



После отправки цвета – фон активности становится выбранного цвета, а в центре появляется надпись вида «Выбран цвет: #RRGGBB», где RRGGBB – это hex-код выбранного цвета:



Ход работы

1. Создан проект Lab20 на основе Empty Views Activity.\

2. В файл манифеста добавлено разрешение на отправку уведомлений и объявлен BroadcastReceiver:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Lab20">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <receiver
            android:name=".NotificationReceiver"
            android:exported="false" />

    </application>
</manifest>
```

3. Настроены ресурсы приложения:

– strings.xml

```
<resources>
  <string name="app_name">Lab20</string>
  <string name="button_show_notification">Показать уведомление</string>
  <string name="notification_title">Управляющий</string>
  <string name="notification_content">Отсюда можно управлять цветом фона
программ.</string>
  <string name="action_reset_color">СБРОСИТЬ ЦВЕТ</string>
  <string name="action_set_color">ЗАДАТЬ ЦВЕТ</string>
  <string name="remote_input_label">Цвет в формате RRGGBB</string>
  <string name="channel_name">Управление цветом</string>
  <string name="channel_description">Канал для управления цветом фона</string>
  <string name="color_selected">Выбран цвет: #s</string>
  <string name="invalid_color_format">Неверный формат цвета</string>
  <string name="hello_world">Hello World!</string>
</resources>
```

– dimens.xml

```
<resources>
  <dimen name="text_size_medium">18sp</dimen>
  <dimen name="margin_bottom_large">32dp</dimen>
</resources>
```

4. Разработан пользовательский интерфейс:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/mainLayout"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:background="#FFFFFF"
  tools:context=".MainActivity">

  <TextView
    android:id="@+id/colorTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/hello_world"
    android:textSize="@dimen/text_size_medium"
    android:visibility="gone"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

  <Button
    android:id="@+id/notificationButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/button_show_notification"
    android:layout_marginBottom="@dimen/margin_bottom_large"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

5. Реализована основная активность MainActivity:

```
package ru.olegkravtsov.lab20

import android.Manifest
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Intent
import android.graphics.Color
import android.os.Build
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.activity.result.contract.ActivityResultContracts
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import androidx.core.app.RemoteInput
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {

    private lateinit var notificationButton: Button
    private lateinit var colorTextView: TextView
    private lateinit var notificationManager: NotificationManagerCompat

    private val requestPermissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted ->
        if (isGranted) {
            showNotification()
        }
    }

    companion object {
        const val CHANNEL_ID = "lab20_channel"
        const val NOTIFICATION_ID = 1
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.mainLayout)) { v,
insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        initView()
        createNotificationChannel()
        handleIntent(intent)
    }

    private fun initView() {
        notificationButton = findViewById(R.id.notificationButton)
        colorTextView = findViewById(R.id.colorTextView)
    }
}
```

```

notificationManager = NotificationManagerCompat.from(this)

notificationButton.setOnClickListener {
    checkNotificationPermission()
}

private fun checkNotificationPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ActivityCompat.checkSelfPermission(
            this,
            Manifest.permission.POST_NOTIFICATIONS
        ) != android.content.pm.PackageManager.PERMISSION_GRANTED
        ) {
requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
            } else {
                showNotification()
            }
        } else {
            showNotification()
        }
    }

private fun createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            CHANNEL_ID,
            getString(R.string.channel_name),
            NotificationManager.IMPORTANCE_DEFAULT
        ).apply {
            description = getString(R.string.channel_description)
        }

        val notificationManager =
getSystemService(NotificationManager::class.java)
        notificationManager.createNotificationChannel(channel)
    }

private fun showNotification() {
    val remoteInput =
RemoteInput.Builder(NotificationReceiver.KEY_TEXT_REPLY).run {
        setLabel(getString(R.string.remote_input_label))
        build()
    }

    val replyIntent = Intent(this, NotificationReceiver::class.java).apply {
        action = NotificationReceiver.ACTION_SET_COLOR
    }

    val replyPendingIntent = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S)
{
        PendingIntent.getBroadcast(
            this,
            0,
            replyIntent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_MUTABLE
        )
    } else {
        PendingIntent.getBroadcast(
            this,
            0,
            replyIntent,

```

```

        PendingIntent.FLAG_UPDATE_CURRENT
    )
}

val setColorAction = NotificationCompat.Action.Builder(
    R.drawable.ic_launcher_foreground,
    getString(R.string.action_set_color),
    replyPendingIntent
).addRemoteInput(remoteInput).build()

val resetIntent = Intent(this, NotificationReceiver::class.java).apply {
    action = NotificationReceiver.ACTION_RESET_COLOR
}

val resetPendingIntent = PendingIntent.getBroadcast(
    this,
    1,
    resetIntent,
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
)

val resetColorAction = NotificationCompat.Action.Builder(
    R.drawable.ic_launcher_foreground,
    getString(R.string.action_reset_color),
    resetPendingIntent
).build()

val mainIntent = Intent(this, MainActivity::class.java).apply {
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
}

val pendingIntent = PendingIntent.getActivity(
    this,
    2,
    mainIntent,
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
)

val notification = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_launcher_foreground)
    .setContentTitle(getString(R.string.notification_title))
    .setContentText(getString(R.string.notification_content))
    .setContentIntent(pendingIntent)
    .setAutoCancel(true)
    .addAction(resetColorAction)
    .addAction(setColorAction)
    .build()

notificationManager.notify(NOTIFICATION_ID, notification)
}

private fun handleIntent(intent: Intent?) {
    when (intent?.getStringExtra("action")) {
        "reset" -> resetColor()
        "set_color" -> {
            val color = intent.getStringExtra("color")
            color?.let { setColor(it) }
        }
    }
}

private fun resetColor() {
    findViewById<androidx.constraintlayout.widget.ConstraintLayout>(R.id.mainLayout)

```

```
        .setBackgroundColor(Color.WHITE)
        colorTextView.visibility = android.view.View.GONE
    }

    private fun setColor(colorHex: String) {
        try {
            val color = Color.parseColor("#$colorHex")

            findViewById<androidx.constraintlayout.widget.ConstraintLayout>(R.id.mainLayout)
                .setBackgroundColor(color)

            colorTextView.text = getString(R.string.color_selected, colorHex)
            colorTextView.visibility = android.view.View.VISIBLE
        } catch (e: IllegalArgumentException) {
            Toast.makeText(this, getString(R.string.invalid_color_format),
                Toast.LENGTH_SHORT).show()
        }
    }

    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        handleIntent(intent)
    }
}
```


7. Реализация NotificationReceiver:

```
package ru.olegkravtsov.lab20

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import androidx.core.app.RemoteInput

class NotificationReceiver : BroadcastReceiver() {
    companion object {
        const val ACTION_RESET_COLOR = "ru.olegkravtsov.lab20.RESET_COLOR"
        const val ACTION_SET_COLOR = "ru.olegkravtsov.lab20.SET_COLOR"
        const val KEY_TEXT_REPLY = "key_text_reply"
    }

    override fun onReceive(context: Context, intent: Intent) {
        when (intent.action) {
            ACTION_RESET_COLOR -> {
                val mainIntent = Intent(context, MainActivity::class.java).apply {
                    putExtra("action", "reset")
                    flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
                }
                context.startActivity(mainIntent)
            }
            ACTION_SET_COLOR -> {
                val remoteInput = RemoteInput.getResultsFromIntent(intent)
                val colorText =
remoteInput?.getCharSequence(KEY_TEXT_REPLY)?.toString()

                if (!colorText.isNullOrEmpty()) {
                    val mainIntent = Intent(context, MainActivity::class.java).apply
{
                        putExtra("action", "set_color")
                        putExtra("color", colorText)
                        flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
                    }
                    context.startActivity(mainIntent)
                }
            }
        }
    }
}
```

Результат работы

Приложение успешно запускается и отображает интерфейс с кнопкой "Показать уведомление" (рис. 1).

11:16

Показать уведомление

Рисунок 1 – Начальный экран приложения

При нажатии на кнопку появляется уведомление (рис. 2).

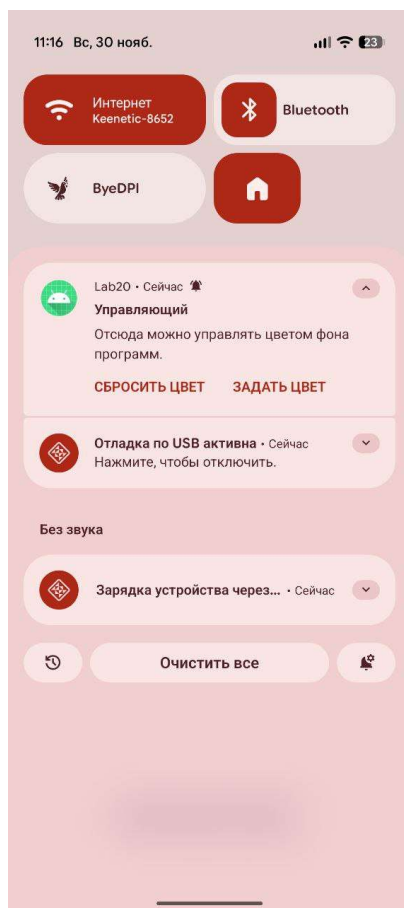


Рисунок 2 – Уведомление

При нажатии на кнопку «ЗАДАТЬ ЦВЕТ» появляется текстовое поле для задания цвета в формате RRGGBB (рис. 3).

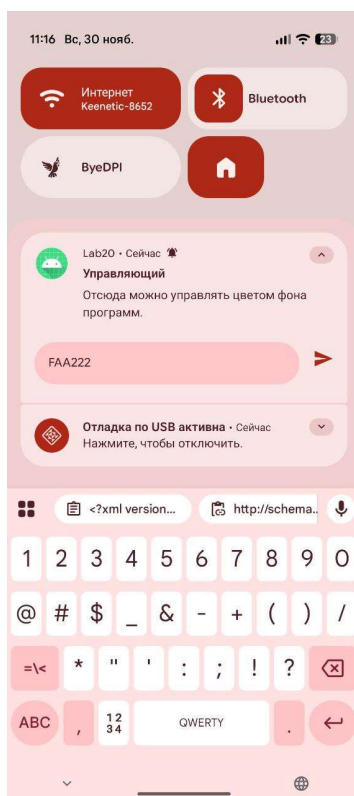


Рисунок 3 – Текстовое поле для задания цвета в уведомлении

После отправки цвета – фон активности становится выбранного цвета, а в центре появляется надпись (рис. 4).



Рисунок 4 – Успешное задание цвета

При нажатии на кнопку «СБРОСИТЬ ЦВЕТ» в уведомлении фон возвращается к белому цвету.

Выводы

В ходе выполнения лабораторной работы было разработано приложение, использующее систему уведомлений Android. Были освоены принципы создания и настройки уведомлений с использованием современного API NotificationCompat, работа с каналами уведомлений для организации группировки и управления поведением оповещений, а также реализация интерактивных действий, включая кнопки и поле ввода текста непосредственно из уведомления.